Resgate Digital — Relatório técnico

Versão 1.0.0. Preparação assistida por IA. Este documento distingue implementação, execução local, evidência pública e participação humana. Fonte primária das evidências: relatórios JSON gerados pelos scripts e execuções do GitHub Actions; nenhum resultado pendente é apresentado como realizado.

1. Proposta, escopo e decisões

Puzzle educativo de programação para iniciantes: controlar um submarino e resgatar alvos em cinco missões. Sequência, direção, repetição, depuração e condição compõem a progressão. Tentativas ilimitadas, dicas, estrelas e pontos oferecem feedback. Jogo em HTML/CSS/JavaScript sem dependências de produção, cadastro ou rede. O ZIP abre index.html em navegador; não exige executável Windows.

Escolhemos comandos visuais e tabuleiro pequeno para reduzir complexidade, manter acesso em celular e computador e permitir testes determinísticos do motor. Código UMD permite execução no navegador offline e testes Node. Separação: src/core para regras, content para missões, web para apresentação, src/ops e scripts para operação. localStorage guarda apenas progresso no dispositivo e oferece exclusão.

2. Equipe e contribuição individual

Integrante / RA	Função
Lucas Souza da Silva / 26179875	Produto e Game Design
Miguel Oliveira de Almeida / 26179933	Desenvolvimento e Qualidade
Davi Marcondes Paes de Souza / 26180842	Plataforma e Release
Lucca Castilho costa / 26179873	SRE e Segurança

SQUAD.md define uma função por pessoa e três etapas por responsável. Há doze snapshots cumulativos. A atribuição nominal de commits foi autorizada pelo usuário, e a preparação com IA está em AI-USAGE.md. Essa atribuição não comprova participação humana em três integráveis. Contribuições, revisões por outra pessoa e retrospectiva pessoal devem ser confirmadas pelos integrantes, conforme docs/CONTRIBUICOES.md.

3. Qualidade e regressões

Cobertura local do motor: 98.75% linhas; 98.2% ramos; 100% funções. Limite exigido: 70%.

Unidade cobre movimentos, colisões, limites, energia, orientação, repetição e condições. Integração valida conteúdo e soluções, loader e publicação. E2E exercita vitórias nas cinco missões, todas as versões, reinício durante execução, celular, funcionamento offline e limpeza dos dados. Falha no CI impede merge da main protegida.

Bug real registrado em docs/REGRESSAO.md: reutilização do diretório Git de publicação falhava ao adicionar origin novamente, afetando rollback no mesmo job. A correção reutiliza a configuração existente; teste cria remoto Git temporário e publica mais de uma vez. Outra regressão cobre sessão com release revogada para migrar o jogador à estável. Evidências são os testes, não uma alegação de indisponibilidade pública.

4. Segurança e autoria

Gitleaks examina o histórico completo e oculta segredos em logs. npm audit bloqueia vulnerabilidade crítica de produção. Dependabot verifica dependências e Actions; SBOM CycloneDX inventaria componentes. LICENSE, THIRD_PARTY.md e AI-USAGE.md identificam licença e assistência de IA. O jogo não coleta nome, e-mail nem dados sensíveis. Credencial GitHub não integra código, ZIP ou site. Testes de alertas são explicitamente rotulados como exercício.

5. Build e release

Reprodutibilidade: duas execuções comparadas; consulte output/relatorios/reprodutibilidade.json para hashes e resultado.

SOURCE_DATE_EPOCH fixa a data no commit; PDFs usam geração determinística e ZIPs usam ordem e timestamps fixos. version.json registra versão, commit do build e commit da etapa. Há checksums SHA-256, GDD por versão e doze ZIPs abaixo de 25 MB. Tags anotadas identificam as etapas históricas; novas alterações operacionais não reescrevem essa autoria nem a data dos commits.

6. Esteira, promoção e recuperação

Push e PR acionam instalação, lint, testes/cobertura, segurança, geração dos PDFs, build e empacotamento. Homologação recebe o artefato aprovado e roda E2E. Produção exige aprovação do ambiente e promove o mesmo artefato. Releases são preservadas por hash; azul-verde troca o ponteiro. Smoke falho aciona rollback automático; workflow manual permite recuperação controlada.

Exercício público de rollback: recuperação medida em 36.183 segundos; aprovado: True; versão original restaurada: True. Workflow: https://github.com/DaviSouza78/resgate-digital/actions/runs/38016229456. Escopo: Demonstração real via workflow protegido rollback.yml; aprovação de ambiente delegada pelo usuário e executada por automação. Sem alegação de revisão humana de PR.. As sondas registradas no JSON verificam HTTP e commit; elas não representam vigilância contínua entre amostras.

7. Observabilidade e DORA

Métricas observadas no painel público: {"deploymentFrequency": 4, "leadTimeMinutes": 6.145833333333333, "changeFailureRate": 0, "recoveryMinutes": null, "method": "Últimas 10 execuções completas: jobs deploy-prd; falha = produção sem sucesso; recuperação = Issues alerta fechadas; frequência com janela mínima de 1 dia."}

Ciclos de homologação: [{"run": "https://github.com/DaviSouza78/resgate-digital/actions/runs/38011212893", "started": "2026-10-10T00:57:13Z", "homologation": "2026-10-10T01:01:01Z", "seconds": 228}, {"run": "https://github.com/DaviSouza78/resgate-digital/actions/runs/38012736380", "started": "2026-10-10T01:19:32Z", "homologation": "2026-10-10T01:21:40Z", "seconds": 128}]

Tempo do acionamento do workflow à homologação aprovada. Comparação observada, incluindo bootstrap inicial; não demonstra causalidade nem garantia de prazo futuro. Decisão: Pages configurado antes dos próximos deploys, lint antecipado e build único; acompanhar os próximos ciclos para confirmar a melhoria. Recuperação sem incidente permanece não medida.

Cron executa sondas a cada 15 minutos; o agendador GitHub pode atrasar. Produção e homologação registram HTTP, latência, versão e commit. HTTP diferente de 200 ou latência maior que dois segundos abre Issue; recuperação fecha automaticamente. Painel /status/ apresenta frequência de deploy, lead time, taxa de falha de mudanças e tempo de recuperação; sem incidente resolvido, recuperação permanece não medida, em vez de zero fictício.

Exercício de alertas: evidência disponível em output/relatorios/alertas-exercicio.json; incidentes locais simulados, sem alegar falha pública.

8. Entrega e comunicação

Vídeo: gameplay real automatizado, legendas visíveis, 68.8 segundos de gravação. A triagem mede a duração do arquivo final.

O pacote de submissão reúne GDD, relatório, URL pública, build.zip, instruções offline, vídeo, legendas e manifesto SHA-256. Triagem confere arquivos, hashes, duração, limite do relatório, equipe e prazo informado pelo usuário: 09/10/2026 às 23:59, fuso America/Montevideo. Entrega posterior ao horário não recebe aprovação automática.

9. Dispensa informada, retrospectiva e limitações

O usuário autorizou entregar e publicar com dispensa das aprovações formais de PR por outro integrante. Registro: delivery-exceptions.json e docs/ENTREGA.md. A informação não foi confirmada independentemente com o professor. Não são declaradas revisões realizadas, contribuição humana não comprovada nem prorrogação do prazo. TRIAGEM.json distingue disponibilidade técnica de conformidade acadêmica integral e registra publicação posterior ao prazo original, se ocorrer.

Aprendizado técnico desta preparação com IA: reutilizar o remoto evita falha de rollback; manter um artefato entre ambientes evita divergência; fixar a data no commit permite comparação de hashes. A decisão de manter as missões pequenas reduziu dependências e permitiu ampla cobertura do motor. Próxima melhoria: acompanhar novos ciclos DORA e entrevistar jogadores para avaliar aprendizagem, sem confundir testes automatizados com avaliação pedagógica.

A equipe deve escrever sua retrospectiva pessoal e confirmar contribuição e revisões. Dez PRs criados não equivalem a dez PRs revisados por outra pessoa. Aprovação obrigatória de produção precisa ocorrer realmente. Evidências de exercício devem registrar execução e tempo, não apenas existência do workflow. Esses limites ficam explícitos no pacote e não são removidos para simular conformidade.